

SYSTEM DESIGN INTERVIEW

Design a Distributed Cache

Memcached / Redis Cluster / DynamoDB-style key-value store at scale

~ 45 minute read

© Study Guide — For personal use only

What's Inside

- 01 · Problem Statement
- 02 · Requirements
- 03 · Capacity Estimation
- 04 · High-Level Architecture
- 05 · Partitioning: Consistent Hashing
- 06 · Replication & Quorum
- 07 · Persistence
- 08 · Eviction Policies
- 09 · Hot Key Problem
- 10 · Cache Stampede & Thundering Herd

- [11](#) · [Data Model & API](#)
- [12](#) · [Failure Modes & Recovery](#)
- [13](#) · [Design Trade-offs](#)
- [14](#) · [Operations & Observability](#)
- [15](#) · [Interview Playbook](#)

01

Problem Statement

Requirements and scope

Design a **distributed in-memory cache / key-value store** comparable to **Memcached**, **Redis Cluster**, or a DynamoDB-style backend. The cache must absorb hot-read traffic for an upstream database, scale horizontally to billions of keys, survive node loss, and offer tunable consistency and durability.

Clarifying Questions

Question	Answer
Workload?	1M ops/sec sustained, mix 80% reads / 20% writes
Working set?	10 TB hot data; total may be much larger but we cache hot subset
Latency target?	p99 < 1 ms intra-DC, < 10 ms cross-AZ
Durability?	Optional: pure cache (lossy) OR primary KV store (persistent)
Consistency?	Tunable: quorum reads/writes (R, W); strong-ish or fastest
Cross-region?	Single region first; async replication for DR
Value sizes?	Mostly < 10 KB; reject > 1 MB at the client
Failure model?	Node, AZ, network partitions; cold restarts

02

Requirements

Functional and non-functional

Functional Requirements

Requirement	Details
GET / SET / DEL	Primitive KV operations on string/binary values
TTL	Per-key expiry; lazy + active eviction
Atomic ops	INCR, CAS (compare-and-swap), SETNX (set-if-not-exists)
Replication	RF=3 with sync or async per shard
Persistence	Optional snapshots (RDB) or append-only log (AOF)
Eviction	Configurable: LRU / LFU / TinyLFU / allkeys-random
Membership	Add / remove nodes online; minimal key movement
Tunable consistency	Per-call R and W quorum ($R+W>N$ for strong-ish)

Non-Functional Requirements

Requirement	Target
Throughput	1M ops/sec sustained, 2M peak
Latency	p50 < 200 us, p99 < 1 ms intra-DC
Cross-AZ p99	< 10 ms (quorum spans AZs)
Availability	99.99% (52 min/yr); survive 1 AZ loss
Durability (KV mode)	RPO < 1 sec with AOF fsync=everysec
Working set	10 TB usable in RAM
Scale-out	Add 1 node → only ~1/N of keys move

03

Capacity Estimation

Sizing the cluster

Throughput Math

- Total: **1,000,000 ops/sec** sustained
- Reads: $80\% \times 1M = 800K \text{ reads/sec}$
- Writes: $20\% \times 1M = 200K \text{ writes/sec}$
- Per-node target: 25K ops/sec (conservative; modern Redis can hit 100K+ on a single core)
- Nodes for throughput: $1M / 25K = \sim 40 \text{ nodes}$; round up to **50** for headroom

Memory Math

- Hot working set: **10 TB** usable
- Per node RAM: **256 GB** (commodity 2-socket box)
- Raw cluster RAM: $50 \times 256 \text{ GB} = 12.8 \text{ TB}$
- After RF=3 replication: $12.8 \text{ TB} / 3 \approx 4.3 \text{ TB usable}$ per copy if every node held a replica
- We instead shard then replicate: **10 TB usable working set** with overhead headroom (slab fragmentation ~15%, replication metadata ~5%)
- Per-key overhead: ~80 bytes (Redis dictEntry + robj + expiry); plan for it

Network Math

- Avg value size: 1 KB; avg op payload incl. protocol: ~1.2 KB
- $1M \text{ ops/sec} \times 1.2 \text{ KB} = 1.2 \text{ GB/sec} \approx 9.6 \text{ Gb/sec}$ aggregate
- Per node: $9.6 / 50 \approx 200 \text{ Mb/sec}$ — well within a 10 GbE NIC
- Replication amplifies writes by RF: $200K \text{ writes} \times 1.2 \text{ KB} \times 2 \text{ followers} = 480 \text{ MB/sec replication traffic}$

KEY

Key Numbers

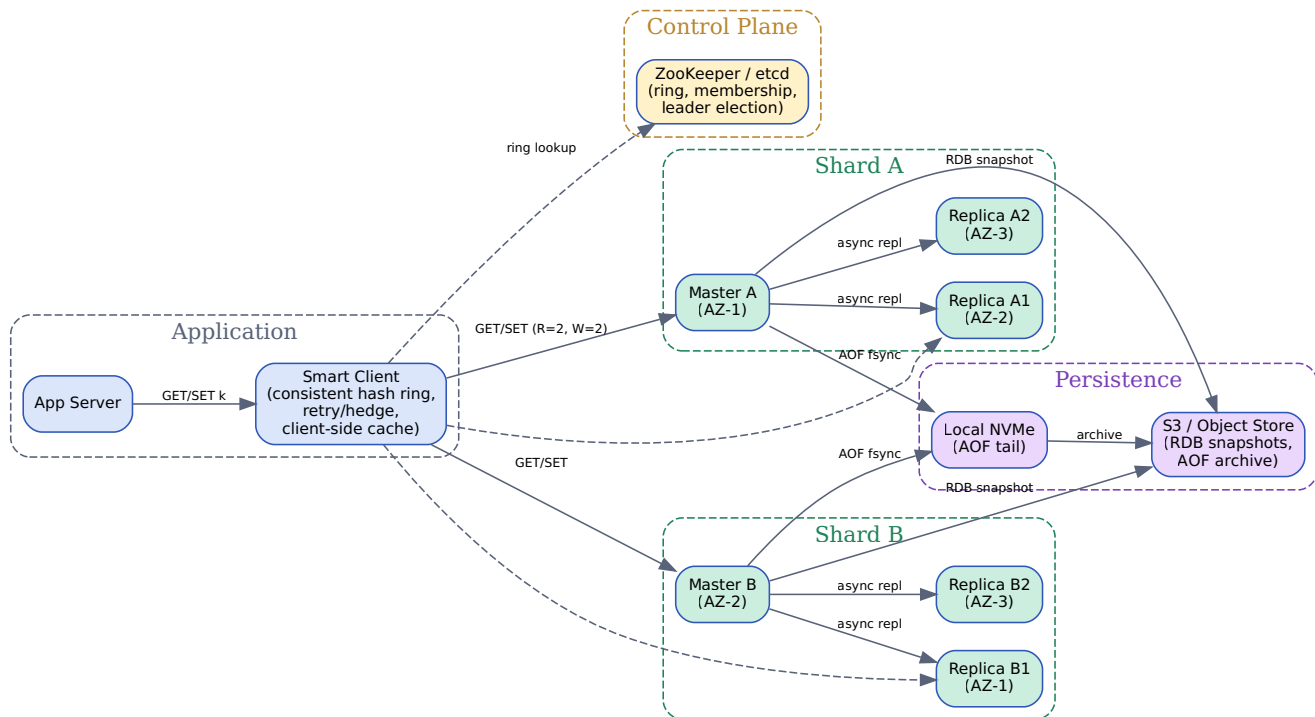
1M ops/sec (80/20 R/W) · **50 nodes** × **256 GB** = **12.8 TB raw** → **10 TB usable** · **RF=3**, R=2 W=2 quorum · **p99 < 1 ms intra-DC**, < 10 ms cross-AZ · consistent hashing with 100-200 vnodes per physical.

04

High-Level Architecture

Smart client → ring → shards

A **smart client** embeds the consistent-hash ring and routes directly to the owning shard. Each shard is a **master + 2 replicas** across AZs. A separate **persistence layer** (S3 / disk) holds RDB snapshots and AOF segments for durability. A **config service** (ZooKeeper / etcd / Redis Sentinel) coordinates membership and failover.



Smart client hashes the key, picks the owning shard, and talks to master + replicas. Persistence is offloaded to S3.

Component Responsibilities

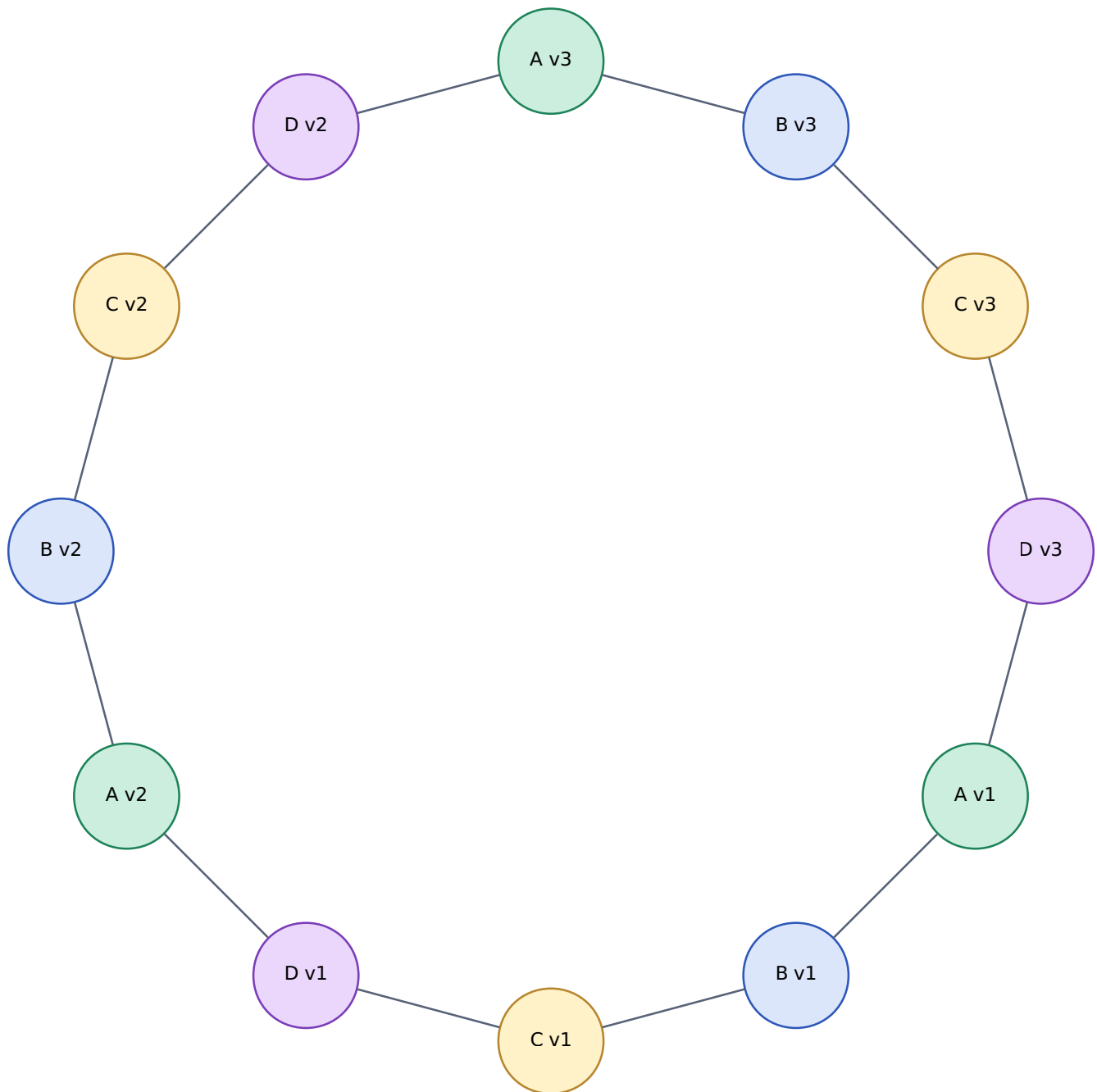
- **Smart client:** embeds ring, hashes keys, retries on timeout, hedges p99-tail reads, optional client-side cache for hot keys
- **Master:** serves writes for its shard; ships changes to replicas
- **Replicas:** serve quorum reads; promotable on master failure
- **Control plane:** ZooKeeper / etcd holds the ring map, runs leader election, gates topology changes
- **Persistence:** NVMe holds the AOF tail; S3 stores RDB snapshots and AOF archives for crash recovery

05

Partitioning: Consistent Hashing

Virtual nodes and the math

Naive $\text{hash}(\text{key}) \% N$ **breaks** when N changes — almost every key remaps. **Consistent hashing** places nodes on a ring ($0 \dots 2^{32}-1$); each key maps to the next node clockwise. Adding/removing 1 node moves only $\sim 1/N$ of keys.



Consistent-hash ring with 4 physical nodes; each placed at multiple positions (vnodes) for load balance.

Why Virtual Nodes (vnodes)

- Without vnodes: 4 random ring positions can land unevenly — one node gets 50% of keys, another 5%
- With **100-200 vnodes per physical**: law of large numbers smooths load to within ~5% of mean
- Resharding: removing node X removes its 100-200 vnodes; their keys flow to neighbours, evenly
- Heterogeneous nodes: a 2× larger box gets 2× the vnodes — capacity-weighted distribution
- Tools: Ketama (libmemcached), Jump Hash (Google), Rendezvous (HRW) — same idea, different math

The 1/N Move-Set Math

```
# Adding one node to a cluster of N
# - new node owns 1/(N+1) of the ring
# - that share comes uniformly from the existing N nodes
# - each existing node loses 1/(N(N+1)) of its keys
# - total keys moved = 1/(N+1) (~ 1/N for large N)

# Example: N = 50 → adding node 51
share_of_new_node = 1 / 51          # ≈ 1.96% of all keys
loss_per_existing = 1 / (50 * 51)  # ≈ 0.039% per old node
# Compare with hash%N which would re-map ~98% of keys.
```

Lookup Pseudocode

```
# Ring is a sorted list of (token, node_id) pairs
# 100-200 tokens per physical node
import bisect, hashlib

def hash64(key: bytes) -> int:
    return int.from_bytes(hashlib.blake2b(key, digest_size=8).digest(), 'big')

def lookup(key: bytes, ring_tokens, ring_nodes, N=3):
    h = hash64(key)
    idx = bisect.bisect_right(ring_tokens, h)
    seen, owners = set(), []
    while len(owners) < N:
        node = ring_nodes[idx % len(ring_tokens)]
        if node not in seen:          # skip extra vnodes of same physical
            seen.add(node)
            owners.append(node)
        idx += 1
    return owners                    # primary, replica1, replica2
```

06

Replication & Quorum

RF=3 with tunable R and W

Replication Factor

- **RF = 3**: every key lives on 1 master + 2 replicas, spread across AZs
- Survives single-node loss **and** single-AZ loss with no data loss
- Cost: 3× storage and 3× write bandwidth — usually worth it for a stateful tier
- RF=2 saves money but loses both copies if both nodes go (e.g., correlated AZ failure)

Quorum Math (Dynamo Style)

Mode	R	W	R+W>N?	Use Case
Strong-ish	2	2	Yes (4>3)	Default; survives 1 replica loss with consistent reads
Read-heavy fast	1	3	Yes (4>3)	Read latency critical; writes can be slower
Write-heavy fast	3	1	Yes (4>3)	Write throughput first; reads pay quorum cost
Fastest (AP)	1	1	No (2<3)	Pure cache; eventual; willing to read stale
Paranoid	3	3	Yes (6>3)	Cannot lose any replica; both ops fail on 1 down

Why R+W > N Gives Strong-ish Reads

- Write touches W nodes; read touches R nodes
- If $R + W > N$, the read set **must intersect** the write set
- At least one node in the read quorum has the latest value
- Use **vector clocks or per-key version** to pick the freshest copy
- Caveat: not linearizable across operations — read-your-writes only if same client sticky-routes

Sync vs Async Replication

Mode	Master Latency	Durability	Failure Risk
Async (default Redis)	Lowest (~ master fsync)	RPO > 0 if master dies first	Dataloss window: ms-seconds
Semi-sync (W=2)		Survives master loss	One follower must ack

Mode	Master Latency	Durability	Failure Risk
	+1 cross-AZ RTT (~ 2-5 ms)		
Sync (W=3)	+max RTT to all replicas	Strongest	Slowest; one slow node blocks all writes

TIP

Recommendation

Default to **RF=3, R=2, W=2**. Tune per workload: cache-mostly traffic can use R=1, W=1 for the lowest latency; financial/idempotency keys use W=3 for absolute write durability.

07

Persistence

RDB snapshots vs AOF append-only log

A pure cache (Memcached-style) **has no persistence** — it's lossy by design. A Redis-style KV store offers two persistence modes which can be combined for belt-and-braces durability.

RDB (Snapshot)

- **How:** periodic `fork()` + serialize entire keyspace to a single binary file
- **Pros:** compact, fast restart (load one file), great for backups → S3
- **Cons:** RPO = snapshot interval (e.g., 5-15 min lost on crash)
- **Fork overhead:** COW pages; large dataset + heavy writes can spike RAM 2× briefly

AOF (Append-Only File)

- **How:** log every write op to disk; replay on restart
- **fsync policies:** always (per write), everysec (1-sec window), no (OS-buffered)
- **Pros:** RPO ≈ 1 sec with everysec; replays exact ops; easy to reason about
- **Cons:** file grows; periodic rewrite/compaction needed; slower restart than RDB

Comparison

Aspect	RDB only	AOF only	RDB + AOF
RPO	Minutes	≈ 1 sec (everysec)	≈ 1 sec
Restart speed	Fast	Slow (replay log)	Fast (RDB) + AOF tail
Disk usage	Low (compact)	High (verbose log)	Medium
Throughput cost	Low (async fork)	Medium (fsync)	Medium
Forensics	Snapshots only	Op-level history	Both

Recommended Setup

```
# redis.conf — production durability
save 900 1          # RDB if 1 key changed in 15 min
save 300 10         # RDB if 10 keys changed in 5 min
save 60 10000       # RDB if 10000 keys changed in 1 min
```

```
appendonly yes
appendfsync everysec # RPO ~ 1 second
auto-aof-rewrite-percentage 100
auto-aof-rewrite-min-size 64mb

# Operational: ship RDB to S3 every 15 min,
# AOF segments to S3 every 5 min (rolling).
```

WARNING

Trade-off: Durability vs Throughput

fsync=always caps writes at the disk's IOPS ($\approx 10\text{K/sec}$ on SSD) — order of magnitude below in-memory rates. **fsync=everysec** is the pragmatic sweet spot: RPO ~ 1 sec, near-RAM throughput. **fsync=no** is fastest but you lose whatever the OS hadn't flushed (seconds to minutes).

08

Eviction Policies

LRU, LFU, TinyLFU, allkeys-random

When a node hits **maxmemory**, the cache must evict something. The policy determines hit rate under churn.

Policies Compared

Policy	What it Evicts	Hit Rate	Cost	Best For
allkeys-random	Any key uniformly	Poor	$O(1)$	Uniform key access (rare)
allkeys-lru	Least recently used	Good	$O(1)$ approx	General purpose; recency-skewed workloads
allkeys-lfu	Least frequently used	Better on hot/cold mix	$O(1)$ approx	Long-tail with stable hot set
volatile-lru	LRU among keys with TTL	Targeted	$O(1)$ approx	Mixed perm + cache keys
volatile-ttl	Soonest to expire	Predictable	$O(\log n)$ heap	TTL-driven workloads
TinyLFU (Caffeine)	Frequency-sketched LRU	Best in benchmarks	Bloom-filter-ish	Modern Java caches; hot-key sparse

Why LRU Is the Default

- Approximates 'likely to be reused soon' from temporal locality
- Redis uses **sampled LRU**: pick K random keys, evict the oldest — $O(1)$, tunable K
- Fails on a one-shot scan (cache pollution): the scan promotes itself, evicts hot data

LFU and TinyLFU

- **LFU**: evict the key accessed least often. Resists scan pollution. Risk: stale 'forever popular' keys never age out
- **Redis LFU**: tracks an 8-bit logarithmic counter per key; counter *decays* over time so old popularity fades
- **TinyLFU (Caffeine library)**: a count-min-sketch frequency filter *in front of* an LRU window. New entries must beat the LFU score of the candidate victim
- TinyLFU uses ~ **1 byte / entry** of metadata; gets ~95% of optimal hit rate on real traces

Sampled LFU Promotion (Pseudocode)

```
# Redis-style approximate LFU on access
# 8-bit log counter; saturates at 255
LFU_LOG_FACTOR = 10 # smaller -> faster ramp-up
LFU_DECAY_MIN = 1 # decay 1 unit per minute idle

def on_access(entry, now_min):
    # decay first
    elapsed = now_min - entry.last_decay_min
    entry.counter = max(0, entry.counter - elapsed * LFU_DECAY_MIN)
    entry.last_decay_min = now_min

    # probabilistic increment (saturating log curve)
    if entry.counter < 255:
        baseval = entry.counter
        p = 1.0 / (baseval * LFU_LOG_FACTOR + 1)
        if random.random() < p:
            entry.counter += 1

def evict_one(sampled): # sampled = K random entries
    return min(sampled, key=lambda e: e.counter)
```

09

Hot Key Problem

When one key gets 10% of all traffic

A celebrity's profile, a product on the homepage, or a flash-sale SKU can suddenly take **10-50% of all reads**. That key lives on one shard — that shard melts. Symptoms: a single CPU pinned at 100%, p99 spikes, queue depth grows.

Mitigations

Strategy	How	Trade-off
Read replicas	Route reads of hot key to replicas (R=1)	Eventual consistency on that key
Hot-key replication	Copy hot key to extra shards; client picks at random	Writes must fan out; staleness window
Client-side cache	Cache hot keys in app process for ~1 sec	Staleness; needs invalidation channel
Request coalescing	Single in-flight fetch per key per node	Adds a wait; reduces backend QPS dramatically
Tiered cache (L1+L2)	Per-app L1 + cluster L2	Memory cost; 2-level invalidation
Sharding the value	Split list/counter into N sub-keys	App must aggregate on read

Detection

- **Server-side:** per-key counter sketches (count-min-sketch over 1-sec windows)
- **Client-side:** sample 1% of requests, bucket by key prefix, alert on outliers
- **Heuristic:** any key > 1% of node QPS is suspect; > 5% is on fire

Hot-Key Replication Pattern

```
# Promote a hot key to N shadow shards.
# Client picks 1 of N at random on read; writes fan out to all N.

HOT_FANOUT = 4 # 4 shadow copies → ~25% load each

def get_hot(key):
    if key in HOT_SET:
        suffix = random.randint(0, HOT_FANOUT - 1)
```

```
        return cache.get(f'{key}#hot{suffix}')
    return cache.get(key)

def set_hot(key, val):
    if key in HOT_SET:
        for i in range(HOT_FANOUT):
            cache.set(f'{key}#hot{i}', val)
    else:
        cache.set(key, val)
```

INSIGHT

Client-side caching is the real win

For read-mostly hot keys, a 1-second client-side cache in front of Redis collapses 100K reads/sec from one app instance into 1 read/sec to the cluster. Redis 6+ supports server-assisted invalidation (CLIENT TRACKING) so the client knows when its local copy goes stale.

10

Cache Stampede & Thundering Herd

When TTL expires for a hot key

A hot key serving 50K reads/sec expires. **50,000 clients** miss in the same millisecond and all stampede the database. The DB melts; the cache stays cold. This is the **thundering herd / cache stampede**.

Three Defences

Technique	How	Best For
Lock-and-fill (singleflight)	First miss takes a Redis SETNX lock; others wait or serve stale	Single-host or coordinated workers
Probabilistic early refresh (XFetch)	Refresh probabilistically before TTL based on remaining time and recompute cost	Best general-purpose; no coordination
Request collapsing	App-server level: combine concurrent identical requests into 1 backend call	Per-process; complements the others

XFetch (Vattani et al., 2015)

Each cached entry stores a **delta** (cost to recompute, in seconds) and an **expiry timestamp**. On every read, decide probabilistically whether to *also* kick off a background refresh. The closer to expiry, the higher the probability.

```
# XFetch probabilistic early refresh
# Stored with each value: delta (recompute cost), expiry
import math, random, time

BETA = 1.0 # higher beta = refresh earlier (more aggressive)

def get_with_xfetch(key, recompute):
    val, delta, expiry = cache.get_with_meta(key)
    now = time.time()
    if val is None or now >= expiry:
        return _refill(key, recompute)
    # probability rises as we approach expiry
    if now - delta * BETA * math.log(random.random()) >= expiry:
        # async background refresh; current request still serves cached val
        spawn(lambda: _refill(key, recompute))
    return val

def _refill(key, recompute):
    # Lock so only one filler runs (singleflight)
```

```
if cache.set_nx(f'lock:{key}', '1', ex=30):
    try:
        t0 = time.time()
        v = recompute()
        delta = time.time() - t0
        cache.set_with_meta(key, v, delta, ttl=300)
        return v
    finally:
        cache.delete(f'lock:{key}')
# someone else is filling; serve stale or wait briefly
return cache.get(key)
```

Lock-and-Fill (Singleflight) Variants

- **Strict lock:** losers block on a short poll → low DB load, higher tail latency
- **Stale-while-revalidate:** losers serve the previous (just-expired) value while the winner refills — best UX
- **Negative caching:** cache 'not found' for a few seconds to absorb miss storms on garbage keys

11

Data Model & API

Redis ops vs DynamoDB-style schema

Redis Command Sequence

```
# Session cache: fast GET / SET with TTL
SET sess:u123 '{"uid":123,"role":"admin"}' EX 3600
GET sess:u123
DEL sess:u123

# Atomic counter – no race even at 1M ops/sec
INCR page_views:home
INCRBY balance:u123 -50

# Compare-and-swap with WATCH/MULTI/EXEC (optimistic)
WATCH inventory:sku-42
MULTI
  DECR inventory:sku-42
  ZADD orders 1714000000 order:9001
EXEC                                # nil if WATCH key changed → retry

# Hot-key fan-out: write to N shards, read 1 at random
MSET hot:product:42#0 v hot:product:42#1 v hot:product:42#2 v hot:product:42#3 v
GET hot:product:42#${random 0..3}

# Stream-processing primitives
XADD events * type click user 123
XREAD COUNT 100 STREAMS events $
```

DynamoDB-Style Table Definition

```
-- Conceptual SQL of a DynamoDB-like KV table
CREATE TABLE kv_items (
  partition_key  VARBINARY(256) NOT NULL,  -- hash → shard
  sort_key       VARBINARY(256) NOT NULL,  -- range within partition
  value          VARBINARY(400000),       -- 400 KB max per item
  version        BIGINT      NOT NULL,    -- for CAS / vector clocks
  ttl_epoch      BIGINT,                  -- background TTL sweeper
  updated_at     TIMESTAMP  NOT NULL,
  PRIMARY KEY (partition_key, sort_key)
);

-- Conditional write (CAS): only succeed if version matches
UPDATE kv_items
```

```

SET value = ?, version = version + 1, updated_at = NOW()
WHERE partition_key = ? AND sort_key = ? AND version = ?;

-- TTL sweeper (lazy + background)
DELETE FROM kv_items WHERE ttl_epoch < UNIX_TIMESTAMP() LIMIT 10000;

```

Memcached vs Redis vs DynamoDB

Aspect	Memcached	Redis Cluster	DynamoDB
Data model	Opaque blob	Strings, lists, hashes, sets, sorted sets, streams	Items with attributes; partition + sort key
Persistence	None (pure cache)	RDB / AOF	Always durable (SSD + RF=3)
Consistency	Per-node only	Async repl by default	Tunable: eventual or strongly consistent reads
Sharding	Client-side (consistent hash)	Built-in slots (16384 hash slots)	Managed; auto-split partitions
Eviction	LRU only	8 policies incl. LRU/LFU	TTL-based; no LRU
Threading	Multi-threaded	Single-threaded per shard	Managed
Best for	Simple LRU object cache	Rich ops, atomic primitives, queues	Durable KV at any scale; pay-per-use

Consistency Models Compared

Model	Guarantee	Example
Linearizable	All ops appear in real-time order; expensive	Spanner, etcd Raft writes
Sequential	Single global order, may lag real time	Multi-Paxos read from leader
Quorum (R+W>N)	Read sees latest acked write; not linearizable	Dynamo, Cassandra, our default
Read-your-writes	Client sees own writes; sticky session	App-level; client tracks last-seen version
Eventual	Replicas converge eventually; reads may be stale	Pure cache (R=1, W=1)

12

Failure Modes & Recovery

What can go wrong (and what we do)

Failure	Impact	Detection	Mitigation
Master node down	Writes fail for that shard until promotion	Heartbeat miss (3 consecutive)	Sentinel/etcd promotes a replica; client reroutes via gossip
Replica node down	Reduced read capacity; possible quorum miss	Health check + lag metric	Re-replicate to spare; bootstrap from RDB + AOF tail
Whole AZ down	1/3 of every shard offline	Cloud provider event + multi-target ping	RF=3 across 3 AZs ensures W=2 still works; degrade to R=2 reads
Network partition (split-brain)	Two nodes both think they're master	Quorum loss in control plane	Fence the loser via STONITH; client rejects writes without lease
Hot shard / hot key	Single CPU pinned; cluster looks idle	Per-shard QPS skew > 5×	Hot-key replication, client-cache, request coalescing (\$09-10)
GC stall / slow swap	p99 latency multi-second; timeouts	p99 vs p50 ratio > 50	Disable swap; pin CPU/NUMA; alert on long GC; jemalloc
Cold restart after crash	Empty cache → DB stampede	Hit rate drops to ~0	Restart from RDB; warm-up replay; rate-limit upstream
Disk full (AOF growth)	Master refuses writes	Disk-usage alert at 80%	Auto-trigger AOF rewrite; ship segment to S3; expand volume

Failover Sequence

1. Sentinel/etcd loses heartbeat from master M for > 3× interval (e.g., 9 sec)
2. Quorum of sentinels agree M is down (avoids false positives from network blips)
3. Pick replica with the highest replication offset (most up-to-date)
4. Promote: SLAVE0F NO ONE; reconfigure other replicas to follow it
5. Update ring map in etcd; gossip new master to clients
6. Smart clients receive the new map (push or next request) and reroute
7. Old master, on rejoin, becomes a replica of the new master after sync

WARNING

Split-Brain Is the Real Killer

Two masters writing the same key = silent data divergence. Always use a **quorum-based control plane** (etcd / ZooKeeper / Sentinel with majority) so only one side of a partition can elect a master. The minority side returns errors — better than corrupting state.

13

Design Trade-offs

Decisions and rationale

Decision	Choice	Trade-off
CAP positioning	AP with quorum (favor availability)	Brief staleness on partition; we lose linearizability but stay up. CP would refuse writes during partition.
Replication factor	RF = 3	3× cost; survives single-AZ loss. RF = 2 saves 33% but two correlated failures = data loss.
Quorum	R = 2, W = 2	Strong-ish reads, ~5 ms cross-AZ latency. R=1/W=1 halves latency but reads can be stale.
Persistence	RDB + AOF everysec	RPO ≈ 1 sec; ~10% throughput cost vs no-persistence. fsync=always is safer but 10× slower.
In-memory vs SSD	RAM-only with cold tier in S3	\$\$\$ / GB high; sub-ms p99. SSD-backed (Aerospike) is 5× cheaper but 5-10× higher p99.
Eviction policy	allkeys-lfu	Resists scan pollution; keeps hot keys hot. Pure LRU is simpler but vulnerable to one-shot scans.
Topology mgmt	Smart client + ring in etcd	Client complexity; lowest hop count. Proxy layer (twemproxy) is simpler but adds 1 RTT and a SPOF.
Hot key strategy	Detect + fan-out + client-cache	Code complexity; saves the cluster. Doing nothing means one shard melts under any virality.
Hash function	Blake2b (64-bit truncated)	Fast (~1 GB/s), uniform. CRC32 is faster but more skew; SHA1 is overkill.
Vnode count	150 per physical	Smooths load to ±5%; ~7.5K ring entries for 50 nodes (cheap to scan/serialize).

Key SLIs to Monitor

- **Hit rate** per node and global: alert if drops > 5% in 1 min
- **Latency**: p50, p99, p999 — separately per shard (catches hot shards)
- **Memory**: used / maxmemory ratio per node; evicted_keys/sec
- **Replication lag**: master_repl_offset - replica_offset (bytes)
- **Connection count**: too many clients → file-descriptor exhaustion
- **Slow-log**: any command > 10 ms in a sub-ms cache is a red flag
- **Per-key QPS skew**: top-K keys via count-min-sketch sampling

Capacity Planning

- Trigger scale-out at **70% memory** or **60% CPU** per node
- Adding a node = ~1/N rebalance; throttle migration to 50 MB/s/node to protect QPS
- Plan for 2× peak headroom — viral events spike 5-10× normal
- Pre-create shard slots at cluster init (e.g., 1024 logical shards on 50 physical nodes) to make later splits cheap

Rolling Upgrades

1. Drain replica B1 (mark not-ready in etcd; clients stop sending it reads)
2. Stop replica; upgrade binary; restart; let it catch up via AOF tail + replication
3. Verify replication lag < 1 s and hit rate stable
4. Repeat for replica B2
5. **Failover**: demote master B; promote B1; upgrade old master B; rejoin as replica
6. Move on to shard A — never upgrade more than 1 replica per shard at once

Backup & Restore

- **RDB snapshot** → **S3** every 15 min; 30-day retention
- **AOF segments** → **S3** every 5 min; rolling
- Restore drill quarterly: spin up cluster from S3, verify hit rate, kill it
- Cross-region async replication for DR (+ 100-500 ms lag)

TIP

The 4 Golden Signals (Cache Edition)

Latency (p99 by shard) · **Traffic** (ops/sec, R:W ratio) · **Errors** (timeouts, MOVED redirects, OOM evictions) · **Saturation** (memory %, CPU %, replication lag). Dashboard them per-shard, not just cluster-wide — averages hide hot shards.

15

Interview Playbook

How to present this

A 45-minute interview on distributed cache design typically rewards depth on **partitioning math**, **quorum logic**, **hot-key handling**, and **stampede prevention**. Drive the conversation through these four pillars.

45-Minute Interview Outline

1. **Clarify (3 min)**: ops/sec, working set, latency, durability requirements
2. **Capacity (5 min)**: 1M ops/sec, 10 TB → 50 nodes × 256 GB, RF=3
3. **Partitioning (7 min)**: consistent hashing with vnodes; show $1/N$ math
4. **Replication & quorum (6 min)**: RF=3, R=2 W=2; why $R+W>N$
5. **Persistence (4 min)**: RDB vs AOF; everysec sweet spot
6. **Eviction (3 min)**: LRU vs LFU vs TinyLFU
7. **Hot keys & stampede (8 min)**: detection, fan-out, XFetch, singleflight
8. **Failures (5 min)**: AZ loss, split-brain, hot shard, cold restart
9. **Wrap (4 min)**: trade-offs explicit; what you'd do differently with more time

Common Follow-Ups

- **Q: Why not hash%N?** A: ~98% of keys remap when N changes; consistent hashing limits movement to $\sim 1/N$.
- **Q: How many vnodes per node?** A: 100-200. Below 50, load is uneven; above 500, ring metadata overhead grows without benefit.
- **Q: Strong consistency?** A: $R+W>N$ gives strong-ish quorum reads; for true linearizability use a Raft-based KV (etcd) or Spanner-style — at cost of latency.
- **Q: Cache vs DB consistency?** A: Cache-aside on read; on write, update DB then DEL cache (not SET — avoids stale-write race). Or write-through if app can tolerate the cost.
- **Q: Defend against thundering herd?** A: Three layers: XFetch probabilistic refresh, lock-and-fill singleflight, request collapsing in app server.
- **Q: How do you find a hot key?** A: Per-shard count-min-sketch over a 1-sec window; alert if any key exceeds 5% of node QPS.
- **Q: What if a master and one replica die together?** A: With RF=3 across 3 AZs, surviving replica becomes master; quorum drops to W=1 until rebuild — accept the risk window.

Talking Points That Score

- **'1/N keys move'** — proves you understand consistent hashing

- '**R + W > N**' — quorum reasoning in one inequality
- '**fsync=everysec is the sweet spot**' — durability vs throughput trade
- '**TinyLFU beats LRU on real traces**' — modern eviction awareness
- '**XFetch + singleflight**' — stampede defence in depth
- '**AZ-aware replica placement**' — survives correlated failure
- '**Smart client cuts a hop**' — vs proxy-based architectures

KEY

Memorize These Numbers

1M ops/sec @ 80/20 R/W · **50 nodes × 256 GB = 12.8 TB raw → 10 TB usable** · **RF=3, R=2, W=2**
 (R+W > N) · **p99 < 1 ms intra-DC, < 10 ms cross-AZ** · **150 vnodes/node** · adding 1 node moves **1/(N+1) ≈ 2%** of keys · **fsync=everysec, RPO ≈ 1 s** · TinyLFU ~ **1 byte/entry**, ~95% of optimal hit rate.